Iqra University, Islamabad

Department of Artificial Intelligence & Computer Science

Machine Learning
AIN-373

PROJECT PROPOSAL — SPRING 2026

Website Quality & Conversion Gap Scorer

An ML-Driven Framework for Automated Website Evaluation, Multi-Dimensional Scoring, and Explainable Recommendations

SUBMITTED BY

Aleem Ul Hassan — 42490

TEAM MEMBER

Abdul Hadi Minhas — 37520

SUBMITTED TO

MS. Khadija Perwez

COURSE

Machine Learning (AIN-373)

PROGRAM

BS Artificial Intelligence — 6th Semester

SUBMISSION DATE

May 1, 2026

SECTION 1

Abstract

The rapid proliferation of online businesses has underscored the critical role of website quality in user retention and conversion optimization. Despite this importance, website quality assessment remains largely manual, subjective, and inconsistent — relying on expensive expert audits or limited single-metric tools such as Google Lighthouse, which capture only technical performance and miss the broader determinants of user trust and conversion behavior.

This project proposes an **ML-driven Website Quality and Conversion Gap Scorer** — an automated system that accepts any website URL, crawls and extracts a rich set of structural, content, and performance features, and applies trained ensemble machine learning models to generate a composite quality score (0–100) across four dimensions: UX design, content readability, technical performance, and conversion optimization.

The system integrates **SHAP-based explainability** to surface the specific features most responsible for a site's score, translating model output into prioritized, actionable recommendations a non-technical user can implement. A **React-based frontend** provides an accessible interface where any URL can be scored in under 30 seconds.

The project targets a dataset of 300–500 manually labeled websites, benchmarks XGBoost, Random Forest, and Gradient Boosting against a rule-based baseline, and evaluates model performance using MAE, RMSE, R^2, F1-Score, and ROC-AUC. Expected deliverables include a deployable scoring tool, a reusable web feature extraction pipeline, and a comparative evaluation of ML approaches for automated web quality assessment.

Keywords: Website quality scoring, ensemble learning, XGBoost, SHAP explainability, feature engineering, web crawling, conversion optimization, usability evaluation

SECTION 2

Problem Statement

Millions of businesses operate websites as their primary digital storefront, yet most lack the technical expertise to objectively assess whether their site is designed to convert visitors into customers. Manual website auditing — performed by UX professionals or digital agencies — is time-consuming (typically 4–8 hours per site), expensive, and inherently subjective. It does not scale to the volume of websites that need evaluation.

Existing automated tools address only narrow slices of the problem:

- **Google Lighthouse / PageSpeed Insights** — measures technical performance (load speed, Core Web Vitals) but does not evaluate UX design quality, persuasive copy, trust signals, or conversion-oriented layout.
- **SEO crawlers (Ahrefs, Semrush)** — focus on search ranking factors and ignore the on-page experience that drives conversion after the visitor arrives.
- **Heuristic checklists** — static rule-based systems that do not learn from patterns in real website data and cannot adapt to industry or audience context.

The consequence is a measurable business cost. Research indicates that a 1-second delay in page load time reduces conversions by approximately 7% [Google, 2020]. Poorly structured landing pages with unclear calls-to-action and low trust signals can suppress conversion rates by 30–50% relative to well-optimized equivalents. Small businesses and agencies building client sites currently have no fast, data-driven way to diagnose these gaps.

> **The core problem:** There is no ML-powered system that learns from labeled examples what distinguishes high-quality, high-converting websites from poor ones, then applies that knowledge at scale with explanations non-experts can act on.

Research Gap

While academic literature on automated usability evaluation exists [Ivory & Hearst, 2001], practical systems that combine multi-dimensional feature extraction, supervised ML scoring, and SHAP-level explainability into a deployable web tool remain absent in open tooling. This project addresses that gap.

SECTION 3

Project Objectives

This project aims to achieve the following objectives:

1. **Build a robust web crawling and feature extraction pipeline** capable of processing any publicly accessible URL, handling both static and JavaScript-rendered pages, and extracting ~40 structured features spanning UX, content, technical, and conversion dimensions.
2. **Engineer a labeled dataset of 300–500 websites** using a standardized scoring rubric, validated with inter-rater reliability measurement (Cohen's Kappa), and cross-referenced with proxy metrics (Google PageSpeed Insights API).
3. **Train and comparatively evaluate multiple ML models** — XGBoost, Random Forest, and Gradient Boosting — to produce a composite website quality score (0–100) and four dimension-level sub-scores, selecting the best model based on MAE and F1-Score.
4. **Integrate SHAP-based explainability** to generate per-website feature contribution analysis, enabling the system to produce prioritized, actionable recommendations tied directly to the model's decision rationale.
5. **Deploy a React-based web interface** that allows any user — including non-technical business owners — to score a website by entering a URL and receive a structured report within 30 seconds.

SECTION 4

Literature Review

4.1 Web Quality Metrics and Standards

Google's Core Web Vitals initiative [Google, 2020] established Largest Contentful Paint (LCP), First Input Delay (FID), and Cumulative Layout Shift (CLS) as measurable proxies for user experience quality, embedding them into search ranking signals. While these metrics are technically rigorous, they capture only rendering and interactivity performance — they say nothing about the persuasive quality of the content, the visual hierarchy of the page, or the psychological trust signals that drive conversion decisions.

4.2 Web Credibility and Trust

Fogg et al. [2002] conducted large-scale studies identifying the primary determinants of perceived website credibility: design quality, information architecture, contact transparency, and social proof. Their findings confirm that trust signals (testimonials, client logos, contact information) are not optional aesthetic choices but measurable predictors of user behavior — making them appropriate engineered features for an ML quality model.

4.3 Readability as a Quality Signal

Flesch [1948] introduced the Flesch Reading Ease score as a standardized formula measuring text complexity based on syllable count and sentence length. Higher readability correlates with longer session duration and lower bounce rates in web contexts. The Flesch-Kincaid Grade Level variant provides an education-level proxy that can be aligned to target audience expectations — making both metrics relevant predictive features in our model.

4.4 Automated Usability Evaluation

Ivory and Hearst [2001] surveyed the state of automated usability evaluation, cataloguing over 130 methods. Their key finding: hybrid computational approaches combining heuristics with learned models consistently outperform pure rule-based systems. This motivates our decision to train an ML model rather than rely on a weighted heuristic checklist, while still incorporating domain-driven feature engineering.

4.5 Ensemble Learning Methods

Breiman [2001] introduced Random Forests as a robust ensemble method combining multiple decision trees with bagging and random feature selection, achieving strong generalization on

structured tabular data. Chen and Guestrin [2016] formalized XGBoost as a scalable gradient boosting framework using second-order Taylor approximations and regularization — achieving state-of-the-art performance on tabular benchmarks. Both methods provide native feature importance scores, making them suitable for interpretability-sensitive applications.

4.6 Explainable AI (SHAP)

Lundberg and Lee [2017] proposed SHAP (SHapley Additive exPlanations), grounding feature attribution in cooperative game theory (Shapley values) to produce consistent, locally accurate explanations for any ML model's predictions. Unlike earlier methods (LIME, permutation importance), SHAP provides both local explanations (why did this site score 43?) and global explanations (which features matter most overall?), making it ideal for a recommendation engine that must explain specific, actionable gaps to end users.

SECTION 5

Proposed Methodology

5.1 Data Collection

The dataset will consist of 300–500 publicly accessible websites sampled across diverse industries (e-commerce, SaaS, professional services, agencies, portfolios) to ensure generalizability. Websites will be sourced from directories, Google search results across industry-specific queries, and a curated pool of client sites.

Crawling stack:

- Static pages: `aiohttp` + `BeautifulSoup4` for async HTML extraction
- JavaScript-rendered pages: `Playwright` for headless browser rendering
- Performance data: Google PageSpeed Insights API for LCP, CLS, TBT, and overall score

Labeling strategy:

- Both team members independently score each website on a standardized 4-dimension rubric (UX, Content, Technical, Conversion — each 0–25), producing a composite label (0–100)
- Inter-rater reliability measured with Cohen's Kappa; disagreements above threshold resolved via discussion
- Technical dimension cross-validated against Google PageSpeed Insights API score
- Estimated labeling effort: ~5 minutes/site × 400 sites = ~33 person-hours total

5.2 Feature Engineering

Approximately 40 features will be engineered across four dimensions. Features are designed to be extractable from HTML + HTTP response data without requiring user behavior data (which would not be available for arbitrary URLs).

UX & Layout (10 features)

- CTA button count (above/below fold)
- Primary CTA visibility in viewport
- Navigation link count
- Navigation depth (max levels)
- Form field count
- Image-to-text ratio
- Estimated whitespace ratio
- Mobile viewport meta tag presence
- Responsive CSS breakpoint count
- Hero section detected (boolean)

Content & Readability (10 features)

- Flesch Reading Ease score
- Flesch-Kincaid Grade Level
- Homepage word count
- Average sentence length
- Heading hierarchy score (H1→H2→H3 structure)
- H1 tag presence and uniqueness
- Value proposition phrase detected
- Primary keyword density
- Content freshness (last-modified header)
- Paragraph length variance

Technical Performance (10 features)

- Google PageSpeed score (mobile)
- Largest Contentful Paint (LCP)
- Total Blocking Time (TBT)
- Cumulative Layout Shift (CLS)
- HTTPS / SSL presence
- Schema markup (structured data) presence
- Image alt text coverage ratio
- Broken link count (sampled)
- External CSS/JS file count
- Total page size (KB)

Trust & Conversion (10 features)

- Testimonial/review section presence
- Client logo section presence
- Social proof count indicator
- Phone number visibility
- Email address visibility
- Physical address presence
- Privacy policy link presence
- Social media link count
- Video content presence
- Pricing page detected

5.3 Machine Learning Pipeline

Three ensemble models will be trained and evaluated: **XGBoost**, **Random Forest**, and **Gradient Boosting (scikit-learn)**. The task is framed as both regression (predict composite score 0–100) and classification (predict quality tier: Poor / Average / Good / Excellent).

Stage	Details
Preprocessing	StandardScaler for continuous features; OrdinalEncoder for boolean/ordinal; MinMaxScaler for PageSpeed scores; median imputation for missing values

Stage	Details
Class Imbalance	SMOTE oversampling on training set; `class_weight='balanced'` in tree models
Hyperparameter Tuning	GridSearchCV with stratified 5-fold cross-validation; tuning: n_estimators, max_depth, learning_rate, subsample
Train/Val/Test Split	60% / 20% / 20% stratified split
Baseline Comparison	Rule-based scorer: weighted sum of PageSpeed score + readability + CTA presence
Final Selection	Best model chosen by lowest MAE on validation set; final evaluation on held-out test set

5.4 Explainability & Recommendation Engine

SHAP's `TreeExplainer` (exact, efficient for tree-based models) will be applied to each prediction to compute per-feature Shapley values. These values drive two outputs:

- **Waterfall chart:** visual breakdown of which features pushed the score up or down from the expected baseline, displayed in the frontend for each scored website
- **Actionable recommendations:** top negative SHAP contributors are mapped to pre-written recommendation templates (e.g., if CTA-above-fold SHAP value is strongly negative → "Add a primary action button in the hero section. Studies show above-the-fold CTAs improve click-through rates by 20–40%.")

Sub-scores for each of the four dimensions (UX, Content, Technical, Conversion — each 0–25) will be produced by summing SHAP contributions within each feature group, enabling targeted diagnosis.

5.5 System Architecture

The system is built as a decoupled three-layer architecture:

SECTION 6

System Architecture

Layer	Technology	Responsibility
Frontend	React 18, Tailwind CSS, recharts	URL input, score visualization, SHAP waterfall chart, recommendations list
Backend API	FastAPI (Python), uvicorn	Request routing, pipeline orchestration, model inference, SHAP computation, JSON response
Crawler	aiohttp, BeautifulSoup4, Playwright	HTML extraction (static + dynamic), HTTP header parsing
External API	Google PageSpeed Insights API	Core Web Vitals (LCP, TBT, CLS, overall score)
ML & XAI	scikit-learn, XGBoost, SHAP, joblib	Preprocessing pipeline, model inference, feature attribution

SECTION 7

Dataset Plan

Parameter	Detail
Target size	300–500 labeled websites
Feature count	~40 engineered features per website
Train / Val / Test	60% / 20% / 20% stratified split
Labeling method	Dual-annotator scoring rubric (0–25 per dimension); Cohen's Kappa ≥ 0.70 target
Quality tiers	Poor (0–40), Average (41–60), Good (61–80), Excellent (81–100)
Imbalance handling	SMOTE on training set; class_weight='balanced' for tree models
Industry coverage	E-commerce, SaaS, professional services, portfolios, agencies
Labeling timeline	Weeks 5–8 (parallel with model development in final weeks)

Labeling Rubric (per dimension, 0–25)

Score	UX	Content	Technical	Conversion
0–10 (Poor)	No clear CTA, confusing nav	Unreadable, generic	Score <50, slow, no HTTPS	No trust signals, no contact
11–17 (Average)	Basic layout, some CTAs	Readable but generic	Score 50–74, some issues	Some signals, weak form

Score	UX	Content	Technical	Conversion
18–25 (Good)	Clear hierarchy, strong CTA	Clear, value-focused copy	Score 75+, fast, secure	Reviews, contact, pricing clear

SECTION 8

Evaluation Metrics

Regression Metrics

- **Mean Absolute Error (MAE)** — primary metric; target: MAE < 8 on a 0–100 scale
- **Root Mean Squared Error (RMSE)** — penalizes large errors
- **R^2 (Coefficient of Determination)** — proportion of variance explained; target: $R^2 > 0.75$

Classification Metrics

- **F1-Score (macro)** — balances precision and recall across all tiers; target: F1 > 0.75
- **Precision & Recall** — per-class breakdown
- **ROC-AUC (one-vs-rest)** — discriminative ability across quality tiers
- **Confusion matrix** — visualize misclassifications between adjacent tiers

Comparison Strategy

- All three ML models vs. rule-based baseline
- 5-fold cross-validated scores (mean ± std) to detect overfitting
- Feature importance comparison across models
- SHAP global summary plots for interpretability analysis

System Evaluation

- End-to-end latency: URL to score in <30 seconds
- Recommendation relevance: manual review of top 3 SHAP-driven suggestions per site
- Crawler robustness: success rate across sampled URLs (target: >90%)

SECTION 9

Project Timeline

Week(s)	Phase	Deliverables	Owner
Phase 1 — Foundation			
1–2	Problem definition & literature review	Finalized feature list, labeling rubric, dataset sources identified	Both
3–4	Web crawler implementation	Working crawler: HTML extraction + PageSpeed API integration + feature extraction script	Abdul Hadi
Phase 2 — Data			
5–6	Dataset labeling (batch 1)	200 websites labeled, inter-rater Kappa calculated, rubric refined	Both
7–8	Dataset labeling (batch 2) & EDA	Full dataset (300–500 sites), exploratory data analysis, preprocessing pipeline	Both
Phase 3 — Modeling			
9–10	Model training & evaluation	XGBoost, RF, GBM trained and benchmarked; best model selected; evaluation report	Aleem

Week(s)	Phase	Deliverables	Owner
11	SHAP integration & recommendation engine	TreeExplainer integrated; waterfall charts generated; recommendation template system	Aleem
Phase 4 — Deployment			
12–13	FastAPI backend & React frontend	Working web app: URL input → score report with SHAP chart and recommendations	Abdul Hadi (FE) / Aleem (API)
14	Integration testing & polish	End-to-end tests, UI refinement, edge case handling (failed crawls, JS-heavy sites)	Both
Phase 5 — Submission			
15	Report & presentation	Final project report, slide deck, demo-ready application	Both

SECTION 10

Team Responsibilities

Aleem Ul Hassan

Student ID: 42490

- Overall system architecture design
- ML pipeline: preprocessing, model training, hyperparameter tuning, evaluation
- SHAP integration and recommendation engine
- FastAPI backend and API endpoint design
- Inter-rater reliability analysis
- Project report & presentation lead

Abdul Hadi Minhas

Student ID: 37520

- Web crawler implementation (aiohttp + BeautifulSoup + Playwright)
- Google PageSpeed Insights API integration
- Feature extraction scripting (all 40 features)
- Dataset collection and labeling (co-annotator)
- React frontend development
- Exploratory data analysis and visualizations

Both team members will collaborate on dataset labeling, with inter-rater reliability evaluated to ensure label quality. Project coordination will be managed via GitHub (version control) and shared Jupyter notebooks for collaborative model experimentation.

SECTION 11

Tools & Technologies

Category	Tools / Libraries	Purpose
Languages	Python 3.11, JavaScript / TypeScript	Backend ML pipeline; frontend interface
ML & Data	scikit-learn, XGBoost, imbalanced-learn, Pandas, NumPy	Model training, preprocessing, evaluation, SMOTE
Explainability	SHAP (shap library)	TreeExplainer, waterfall plots, feature attribution
Web Crawling	aiohttp, BeautifulSoup4, Playwright	Async HTML extraction; JavaScript-rendered pages

Category	Tools / Libraries	Purpose
External API	Google PageSpeed Insights API	Core Web Vitals (LCP, TBT, CLS, overall score)
NLP / Readability	textstat, spaCy	Flesch Reading Ease, Kincaid Grade Level, text features
Backend	FastAPI, uvicorn, joblib, Pydantic	REST API, model serialization, request validation
Frontend	React 18, Tailwind CSS, recharts, Vite	Score dashboard, SHAP waterfall visualization
Experimentation	Jupyter Notebook, Matplotlib, Seaborn	EDA, model evaluation plots, SHAP global summaries
Dev Tools	Git, GitHub, VS Code, pytest	Version control, collaboration, testing

Technology Stack Summary

Python 3.11 | XGBoost | Random Forest | Gradient Boosting | SHAP | scikit-learn | FastAPI | React 18 | Tailwind CSS | BeautifulSoup4 | Playwright | aiohttp | PageSpeed Insights API | textstat | Pandas | NumPy | Jupyter | GitHub

SECTION 12

Expected Deliverables

#	Deliverable	Description
1	Labeled Dataset	CSV file containing 300–500 websites with ~40 features and dual-annotator composite scores (0–100). Will be made available as a course artifact.
2	Feature Extraction Pipeline	Reusable Python module that accepts any URL and returns a structured feature dictionary in <10 seconds (excluding PageSpeed API latency).
3	Trained ML Model	Best-performing model (serialized via joblib) with preprocessing pipeline, cross-validation results, and full evaluation report comparing XGBoost, RF, and GBM against baseline.
4	SHAP Explainability Report	Global SHAP summary plots identifying the most impactful features across the dataset; local waterfall plots for 5 sample websites illustrating per-site explanations.
5	Web Application	Full-stack app (FastAPI + React) where any user can enter a URL and receive a score report with dimension sub-scores, SHAP waterfall chart, and ranked recommendations within 30 seconds.
6	Project Report & Presentation	Final written report (IEEE format) covering methodology, results, analysis, and limitations. 15-minute slide presentation with live demo.
7	GitHub Repository	Public GitHub repo with clean code, README, setup instructions, and reproducible Jupyter notebooks for all experiments.

SECTION 13

References

[1] Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32. https://doi.org/10.1023/A:1010933404324

[2] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). ACM. https://doi.org/10.1145/2939672.2939785

[3] Flesch, R. (1948). A new readability yardstick. *Journal of Applied Psychology, 32*(3), 221–233. https://doi.org/10.1037/h0057532

[4] Fogg, B. J., Soohoo, C., Danielson, D., Marable, L., Stanford, J., & Tauber, E. R. (2002). How do users evaluate the credibility of web sites? A study with over 2,500 participants. In *Proceedings of the 2002 Conference on Designing for User Experiences* (pp. 1–15). ACM. https://doi.org/10.1145/997078.997097

[5] Google. (2020). *Web Vitals: Essential metrics for a healthy site*. web.dev. https://web.dev/vitals/

[6] Ivory, M. Y., & Hearst, M. A. (2001). The state of the art in automating usability evaluation of user interfaces. *ACM Computing Surveys, 33*(4), 470–516. https://doi.org/10.1145/503112.503114

[7] Lundberg, S. M., & Lee, S. I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems* (Vol. 30, pp. 4765–4774). Curran Associates.

[8] Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *Annals of Statistics, 29*(5), 1189–1232. https://doi.org/10.1214/aos/1013203451